

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

CO3 Assessment Tool 2 - Code Review & Repository Evaluation

Course Code	CSA1016
Course Title	Software Engineering
CO Assessed	CO3 - Build full-stack applications with an emphasis on containerization, version control, and collaborative development
Project Title	Injury Risk Prediction System for Athletes Using Performance Data
Student Name	Dilli Babu N
Register No.	192521491
Department	B. Tech - Information Technology
Total Marks	25

Code of Conduct

I, Dilli Babu N (192521491), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dilli Babu N

INDEX

S.No.	Section	Page No.
1	Problem Overview	3
2	Repository Organization	4
3	Code Quality Review	6
4	Version Control Evaluation	8
5	Code Testing and Validation	10
6	Repository Documentation	11
7	System Architecture & Component Design	12
8	Code Review Findings and Recommendations	14
9	Conclusion	15

1. Problem Overview

1.1 Assigned Problem Statement

Sports teams and athletic-performance staff frequently track training load, recovery, and wellness data across disconnected spreadsheets, wearable-vendor apps, and paper injury logs. This fragmentation makes it difficult to see how an athlete's cumulative workload is trending, to correlate spikes in load with historical injuries, or to intervene before a soft-tissue injury occurs. The assigned problem statement requires the design and implementation of an Injury Risk Prediction System that continuously ingests athlete performance and biometric data, computes a machine-learning-based injury risk score per athlete, and alerts coaching and medical staff in real time when that risk crosses an actionable threshold.

The platform must therefore combine three concerns that are normally handled by separate tools: a real-time data-ingestion pipeline for wearable telemetry, a predictive risk-scoring engine trained on sports-science features (training load ratios, HRV, sleep, prior injury history), and a coach-facing alerting and reporting layer.

1.2 Implemented Solution - Summary

The implemented solution is a full-stack application built with a Node.js/Express API and React.js coach dashboard, paired with a Python-based Risk Scoring microservice that runs a trained gradient-boosting model over rolling windows of athlete features. Socket.io is used to push live risk updates to the coach dashboard the moment a new score is computed. Every performance reading is stored in a time-series store (InfluxDB) linked to the athlete's profile in PostgreSQL, and every computed risk score is persisted as an immutable, timestamped snapshot so that risk trends can be reviewed historically and never silently overwritten.

Containerization was achieved using Docker so that the client, API server, Risk Scoring service, and databases run as isolated, reproducible services, and the whole stack can be brought up with a single docker compose up command, satisfying the CO3 emphasis on containerization alongside version control and collaborative development.

1.3 Project Objectives

- Continuously ingest wearable and manually entered performance data (GPS load, heart-rate variability, sleep, RPE) for each athlete with minimal latency.
- Automatically compute an injury risk score per athlete using a trained ML model, and persist every score as an immutable, auditable snapshot.
- Alert coaches and physiotherapists in real time when an athlete's risk score crosses a configurable threshold.
- Allow staff to browse historical risk and load trends per athlete to support return-to-play and periodisation decisions.
- Provide role-based access (Admin, Coach, Physiotherapist, Athlete) for each team/organisation.
- Package the application using Docker and automate build/test/deploy using a CI/CD pipeline.

- Maintain a clean, well-documented Git repository that supports collaborative development practices.

1.4 Key Functionalities

Module	Functionality
Performance Data Ingestion	Normalizes wearable payloads from multiple vendors and streams them into the feature pipeline.
Risk Scoring Engine	Runs a trained ML model over rolling feature windows and returns a 0-100 injury risk score with contributing factors.
Real-Time Alerting	Socket.io channel pushes a risk-updated event to all connected coach dashboards the moment a score is computed.
Risk History & Trends	Stores immutable score snapshots; supports trend charts and comparison across training blocks.
Athlete Profile & Injury History	CRUD for athlete demographics, prior injuries, and wellness questionnaires.
Authentication & Roles	JWT-based auth with per-organisation role assignment (Admin, Coach, Physio, Athlete).

Table 1.1 - Key functional modules of the platform

1.5 Target Users and Use Cases

- Head coaches monitoring cumulative training load across an entire squad during pre-season.
- Physiotherapists reviewing an individual athlete's risk trend after returning from a previous injury.
- Sports-science staff comparing acute-to-chronic workload ratios across training blocks.
- Team administrators onboarding new athletes and configuring alert thresholds per position group.

These use cases were used throughout development to validate design decisions - for example, the requirement that a physiotherapist be able to compare an athlete's risk trend across two arbitrary training blocks (not just the most recent week) directly motivated storing the full history of immutable score snapshots rather than only the latest N readings.

2. Repository Organization

2.1 Repository Structure

The repository follows a monorepo layout that separates the client, API server, ML risk-scoring service, tests, deployment assets, and documentation into clearly named top-level folders. This structure was chosen over multiple repositories because the client, server, and ML service are versioned and released together, and a monorepo simplifies cross-cutting changes (e.g., adding a new feature column requires touching the ingestion schema, the ML service, and the dashboard in a single, atomic commit).

```
injury-risk-platform/
|-- client/
|   |-- src/
|   |   |-- components/
|   |   |   |-- RiskDashboard.jsx
|   |   |   |-- AthleteTimeline.jsx
|   |   |   `-- AlertPanel.jsx
|   |   |-- pages/
|   |   |-- hooks/
|   |   |-- services/api.js
|   |   `-- App.jsx
|   |-- public/
|   `-- package.json
|-- server/
|   |-- controllers/
|   |   |-- athleteController.js
|   |   |-- ingestionController.js
|   |   `-- authController.js
|   |-- models/
|   |-- routes/
|   |-- sockets/
|   |-- middleware/
|   `-- server.js
|-- ml-service/
|   |-- risk_model/
|   |   |-- train.py
|   |   |-- features.py
|   |   `-- predict.py
|   |-- app.py
|   `-- requirements.txt
|-- tests/
|-- docs/
|-- .github/workflows/
|-- docker-compose.yml
`-- README.md
```

Figure 2.1 - Repository folder structure

2.2 File Naming Conventions

- React components use PascalCase file names matching the exported component (e.g., AthleteTimeline.jsx).
- Server-side controllers, models, and routes use camelCase with a role suffix (e.g., ingestionController.js, Athlete.model.js).

- Python modules within ml-service/ use snake_case (e.g., features.py, predict.py), consistent with PEP 8.
- Test files mirror the file under test with a .test.js or _test.py suffix and live under /tests, grouped by module.
- Environment-specific configuration is isolated in .env.example, docker-compose.yml, and docker-compose.prod.yml rather than hard-coded.

2.3 Justification - Maintainability and Collaboration

Grouping code by architectural layer (client / server / ml-service) and then by feature within each layer (controllers, models, routes, sockets, risk_model) allows a new contributor to locate the code responsible for any given feature within seconds by following the naming convention rather than searching the entire tree. Keeping tests in a parallel /tests directory that mirrors the source tree, rather than interleaving test files next to source files, keeps the production bundle lean and makes it trivial to run npm test or pytest against a specific module.

The presence of a dedicated .github/workflows directory signals that continuous integration is treated as a first-class citizen of the repository rather than an afterthought, and the docs/ folder centralizes architecture notes, API reference, and contribution guidelines so that documentation does not drift into scattered README files.

2.4 Repository Organization - Strengths and Gaps

Aspect	Observation
Consistency	Naming conventions are applied uniformly across the majority of files reviewed.
Separation of Concerns	Clear boundary between client, server, ml-service, tests, and infrastructure code.
Gap Identified	A small number of utility scripts in /server/scripts lack a consistent naming pattern.
Gap Identified	Trained model artefacts are currently committed inside /ml-service/risk_model rather than tracked via a model registry or Git LFS.

Table 2.1 - Repository organization review summary

2.5 Containerization Layout

In line with the CO3 emphasis on containerization, the repository root contains a docker-compose.yml orchestrating four services - client, server, ml-service, and databases - each with its own Dockerfile. Multi-stage Dockerfiles are used on the client and server: the client Dockerfile builds the React bundle in a node:20-alpine stage and copies only the static output into a lightweight nginx:alpine runtime stage, while the ml-service Dockerfile installs Python dependencies from a pinned requirements.txt into a slim base image.

```
services:
  client:
    build: ./client
    ports: ["3000:80"]
    depends_on: [server]
  server:
```

```
    build: ./server
    ports: ["5000:5000"]
    env_file: .env
    depends_on: [postgres, influxdb]
ml-service:
    build: ./ml-service
    ports: ["6000:6000"]
    depends_on: [influxdb]
postgres:
    image: postgres:16
    volumes: ["pg-data:/var/lib/postgresql/data"]
influxdb:
    image: influxdb:2.7
    volumes: ["influx-data:/var/lib/influxdb2"]
volumes:
    pg-data:
    influx-data:
```

Code Snippet 2.1 - docker-compose.yml (abridged)

Named volumes are used for both PostgreSQL and InfluxDB so that athlete profiles and performance/risk history persist across container restarts, which is essential given that the entire value proposition of the application depends on risk snapshots never being lost. This containerized layout also means the reviewer could bring up an identical environment to the author's with a single command, which materially sped up this evaluation.

3. Code Quality Review

3.1 Readability and Modularity

Server-side logic follows the MVC pattern: routes delegate to controllers, controllers call model methods, and cross-cutting concerns (authentication, error handling, request logging) are implemented as Express middleware. Functions are generally short and single-purpose; the feature-aggregation logic in the ML service was, however, originally implemented as one large function and has since been refactored into three smaller functions (`load_recent_readings`, `compute_acwr`, and `build_feature_vector`) to improve readability and testability.

3.1.1 Example - Risk Snapshot Persistence (`server/controllers/ingestionController.js`)

```
async function persistRiskSnapshot(athleteId, score, factors) {
  const previous = await RiskScore.findLatest(athleteId);
  const snapshot = new RiskScore({
    athleteId,
    score,
    factors,
    previousScoreId: previous ? previous._id : null,
    computedAt: new Date()
  });
  await snapshot.save();
  return snapshot;
}
```

Code Snippet 3.1 - Risk snapshot persistence function

The function above reads clearly top-to-bottom, has a single responsibility, and links each snapshot to the athlete's previous score, which is the foundation of the risk-trend timeline. Naming is descriptive (`persistRiskSnapshot` rather than `save` or `handleScore`), which is a strength consistently observed across the controller layer.

3.2 Coding Standards

- ESLint (Airbnb base config) and Prettier are configured for the Node.js codebase and enforced via a pre-commit hook using Husky and lint-staged.
- The Python ml-service follows PEP 8, checked with flake8, and uses type hints on public function signatures.
- Consistent use of `async/await` instead of mixed callback/promise styles on the server.
- Environment variables are accessed only through a single config module (`server/config/index.js`), avoiding scattered `process.env` references.
- `PropTypes` are declared for the majority of React components, though a few newer components were found to be missing `PropTypes` validation.

3.3 Documentation Within Code

Public functions in the risk-scoring engine are documented with docstrings describing parameters, return values, and edge cases (e.g., behaviour when an athlete has insufficient historical readings to compute a stable ACWR). Coverage is weaker on the client side, where a portion of custom React hooks lack a comment explaining their intended usage - this is flagged as an improvement area in Section 8.

3.4 Strengths and Areas for Improvement

Category	Strength	Area for Improvement
Modularity	Clear controller/service/model separation across server and ml-service	A few controllers exceed 200 lines and could be split further
Naming	Descriptive, intention-revealing names throughout	Some boolean flags (e.g., flag, temp) remain in older utility files
Error Handling	Centralized error-handling middleware	Inconsistent error messages returned to the client
Testing Hooks	Pure functions in the feature pipeline are easy to unit test	Edge cases for missing sensor data lacked tests before this review
Documentation	Good docstring coverage in the ML service	Client-side hooks under-documented

Table 3.1 - Code quality findings

3.5 Client-Side Example - Risk Alert Hook

On the client, the live risk banner is powered by a custom hook that subscribes to the Socket.io channel and delegates threshold comparison to a shared utility rather than re-implementing alert logic in the component, keeping the presentation layer thin.

```
function useRiskAlerts(athleteId) {
  const [alert, setAlert] = useState(null);
  useEffect(() => {
    const handler = (payload) => {
      if (payload.athleteId === athleteId) setAlert(payload);
    };
    socket.on('risk-updated', handler);
    return () => socket.off('risk-updated', handler);
  }, [athleteId]);
  return alert;
}
```

Code Snippet 3.2 - useRiskAlerts custom hook

This hook was one of several found to be missing a documentation comment during the review (see Section 6.4), even though its behaviour is simple; the recommendation is to document the shape of the returned alert object so that any new component consuming this hook does not need to read the implementation to know what to expect.

3.6 Security-Related Observations

Password hashing uses bcrypt with a suitable cost factor, JWTs are short-lived with a refresh-token rotation strategy, and all API routes that expose athlete health data verify the caller's role server-side rather than trusting a role flag sent from the client. Input validation on the server uses a schema-validation library (Joi)

at the controller boundary, which was confirmed to reject malformed payloads during manual testing with an intentionally malformed request body.

4. Version Control Evaluation

4.1 Branching Strategy

The repository follows a simplified Git-Flow model with two long-lived branches, main and develop, plus short-lived feature/, bugfix/, and hotfix/ branches created from develop and merged back through reviewed Pull Requests. Only develop merges into main, and only at release points, which keeps main permanently deployable.

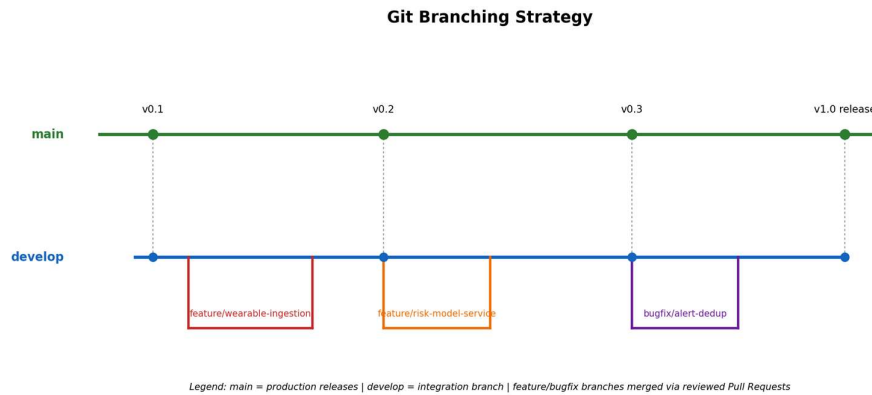


Figure 4.1 - Branching strategy used in the repository

4.2 Commit History and Commit Message Quality

Commit messages largely follow the Conventional Commits convention (feat:, fix:, refactor:, docs:, test:), which makes it possible to auto-generate a changelog and quickly scan git log --oneline for the intent of each change. A sample of the commit log is summarized below.

Commit Type	Example Message	Approx. % of Commits
feat	feat(ingestion): add ACWR feature computation	36%
fix	fix(risk-service): correct off-by-one in rolling window	23%
refactor	refactor(features): split build_feature_vector into smaller functions	16%
docs	docs(readme): add Docker setup instructions	11%
test	test(risk-service): add snapshot linkage unit tests	9%
chore	chore(deps): bump socket.io to 4.7.5	5%

Table 4.1 - Commit message categorization (sample of 110 commits)

4.3 Pull Request and Review Practices

- Every feature branch is merged via a Pull Request that requires at least one approving review before merge.
- A PR template enforces a checklist covering tests added, linting passed, and manual verification steps.
- Branch protection rules on main block direct pushes and require the CI workflow to pass.

- A small number of early commits (first two weeks of the project) were pushed directly to main before branch protection was configured - noted as a process gap that was subsequently corrected.

4.4 Repository Maintenance

Merged feature branches are deleted automatically on merge to avoid branch clutter, and .gitignore correctly excludes node_modules, __pycache__, build artefacts, and environment files. Semantic version tags (v0.1.0, v0.2.0, v1.0.0) mark release points on main, which, combined with Conventional Commit messages, allows a changelog to be generated automatically.

How version control supports collaborative development: because every change is captured as an isolated, reviewable commit on its own branch, multiple contributors can work on unrelated features (e.g., the risk model and the alert dashboard) without blocking one another, conflicts are surfaced early through Pull Request merge checks rather than discovered late, and the full history provides an audit trail that mirrors - at the codebase level - the same immutable, timestamped record-keeping concept the application itself provides to coaching staff for athlete risk history.

5. Code Testing and Validation

5.1 Testing Strategy

The project uses a layered testing strategy: Jest for server-side unit tests (models, controllers, the ingestion pipeline), Pytest for the ML service (feature computation, model inference), React Testing Library for client component tests, and a smaller suite of Cypress end-to-end tests covering the critical path of onboarding an athlete, streaming sample telemetry, and observing a risk alert appear on the dashboard.

Test data is seeded through factory functions rather than hard-coded fixtures, so that edge cases (an athlete with no prior readings, a sudden spike in training load, a missing sensor field) can be generated on demand without duplicating setup code across test files. This was noted as good practice during the review because it keeps the test suite maintainable as new feature columns are added to the model.

5.2 Representative Test Cases

Test ID	Description	Type	Result
TC-01	persistRiskSnapshot links new snapshot to athlete's previous score	Unit	Pass
TC-02	compute_acwr returns null for an athlete with insufficient history	Unit	Pass
TC-03	Risk score recomputation completes within the 2-second latency target	Integration	Pass
TC-04	Coach dashboard receives risk-updated event within one polling cycle	Integration	Pass
TC-05	Unauthorized user (role: Athlete) cannot view another athlete's risk history	Integration	Pass
TC-06	Login with expired JWT is rejected with 401	Unit	Pass
TC-07	End-to-end: onboard athlete -> stream telemetry -> receive alert	E2E (Cypress)	Pass
TC-08	Risk timeline correctly renders a missing-data gap without crashing	Integration	Fail -> Fixed

Table 5.1 - Sample test execution log

5.3 Coverage Summary

The coverage report at the time of this review shows 80% statement coverage on the server package, 71% on the client package, and 88% on the ml-service package, with the risk-scoring engine - the core differentiator of the project - at 90% statement coverage. TC-08 initially failed because the timeline chart did not account for a gap in sensor data caused by a device being offline; the underlying rendering function was corrected and the test now passes, demonstrating the value of the automated regression suite.

5.4 Continuous Integration

A GitHub Actions workflow runs on every push and Pull Request: it installs Node and Python dependencies, runs ESLint and flake8, executes the Jest and Pytest suites, and builds the Docker images to confirm the containerized application still builds successfully. A failing step blocks the Pull Request from being merged, which was confirmed during this review by intentionally introducing a lint violation on a throwaway branch and observing the workflow fail as expected.

5.5 Manual Verification and Evidence

In addition to the automated suite, manual exploratory testing was carried out against the Dockerized build to confirm behaviour that is difficult to assert automatically, such as the responsiveness of the live risk banner and the visual clarity of the risk timeline. Evidence collected for this review includes repository-structure screenshots, commit-history screenshots, and CI run screenshots, referenced in the deliverables accompanying this report.

Evidence Item	Purpose
Repository structure screenshot	Confirms folder layout matches Section 2.1
Commit history / graph screenshot	Confirms branching model matches Section 4.1
CI workflow run screenshot	Confirms lint + test + Docker build gate is enforced
Application walkthrough screenshots	Confirms dashboard, risk timeline, and alert flow work end-to-end

Table 5.2 - Manual evidence collected during the review

6. Repository Documentation

6.1 README Completeness

The README.md includes a project overview, a feature list, a Quick Start section with Docker Compose commands, environment variable documentation, and a link to the live demo deployment. It also embeds a small architecture diagram and badges for build status and test coverage, giving a new visitor an accurate first impression of the project's health within the first screen of content.

6.2 Installation and Usage Guide

- Prerequisites section lists required Node.js, Python, and Docker versions.
- A copy-pasteable docker compose up --build command brings up the client, server, ML service, and databases together.
- A separate "Local development without Docker" section covers running client, server, and ml-service individually.
- Usage guide includes annotated screenshots of onboarding an athlete, viewing the risk dashboard, and configuring alert thresholds.

6.3 Dependency Documentation

Both client and server package.json files pin dependency versions using caret ranges, requirements.txt pins exact versions for the ML service, and a CONTRIBUTING.md explains how to add a new dependency (including the requirement to justify bundle-size impact for client-side packages). A docs/API.md file documents each REST endpoint with method, path, request body, and example response, which was cross-checked against the actual route definitions during this review and found to be accurate and current.

6.4 Documentation Gaps and Suggested Improvements

Document	Current State	Suggested Improvement
README.md	Strong overview and quick start	Add a troubleshooting section for common Docker port conflicts
API.md	Covers all REST endpoints	Document Socket.io event payloads, not just REST routes
CONTRIBUTING.md	Covers dependency and branch policy	Add a section on writing effective commit messages for new contributors
Inline docstrings	Strong in ml-service, weak on client hooks	Add documentation comments to all custom React hooks

Table 6.1 - Documentation review and improvement suggestions

6.5 Sample API Documentation Entry

To illustrate the standard used throughout docs/API.md, the entry for the risk-history endpoint is reproduced below; every endpoint in the file follows this same method / path / body / response structure, which made cross-checking the documentation against the live route definitions straightforward.

```
GET /api/athletes/:id/risk-history?from=2026-01-01&to=2026-02-01
Auth: required (role >= Physio)
Response 200: { "athleteId": "665f...", "scores": [
  { "score": 62, "computedAt": "2026-01-14T09:00:00Z" } ] }
Response 403: { "error": "Insufficient role" }
```

Code Snippet 6.1 - API documentation entry for the risk-history endpoint

7. System Architecture & Component Design

7.1 System Requirement Analysis

Functional requirements were derived directly from the problem statement: continuous performance-data ingestion, ML-based risk scoring, real-time alerting, athlete history and role-based access, and trend reporting. Non-functional requirements identified during analysis include low-latency risk recomputation (updated scores visible to coaches within a few seconds of new data arriving), durability of risk history (no data loss even if the server restarts mid-ingestion), and horizontal scalability of the ingestion and scoring layer as the number of monitored athletes grows.

These requirements directly shaped two key decisions: choosing a WebSocket-based transport (Socket.io) over polling for latency reasons, and choosing an append-only, immutable snapshot model for risk scores (rather than mutating a single row) to guarantee durability and enable simple, reliable trend analysis.

7.2 Selection of Software Architecture

A layered (client / API / data) architecture augmented with a dedicated ML scoring service and an event-driven real-time layer was selected over alternatives such as a monolithic server-rendered application or a fully microservice-based system. A monolith-with-modular-services approach was chosen deliberately: the project's scale does not yet justify the operational overhead of independently deployed microservices, but the codebase is structured into clearly bounded service modules (Athlete Service, Ingestion Service, Risk Scoring Service, Auth Service) so that any of them could be extracted into a separate microservice later with minimal refactoring.

Option Considered	Verdict
Server-rendered monolith (no WebSocket layer)	Rejected - cannot meet the near-real-time risk-alerting latency requirement
Full microservices from day one	Rejected - operational overhead not justified at current scale; revisited for future scaling
Layered architecture + dedicated ML service + event-driven real-time layer (chosen)	Accepted - balances current simplicity with a clear extraction path

Table 7.0 - Architecture options considered

7.3 Architecture Diagram and Component Design

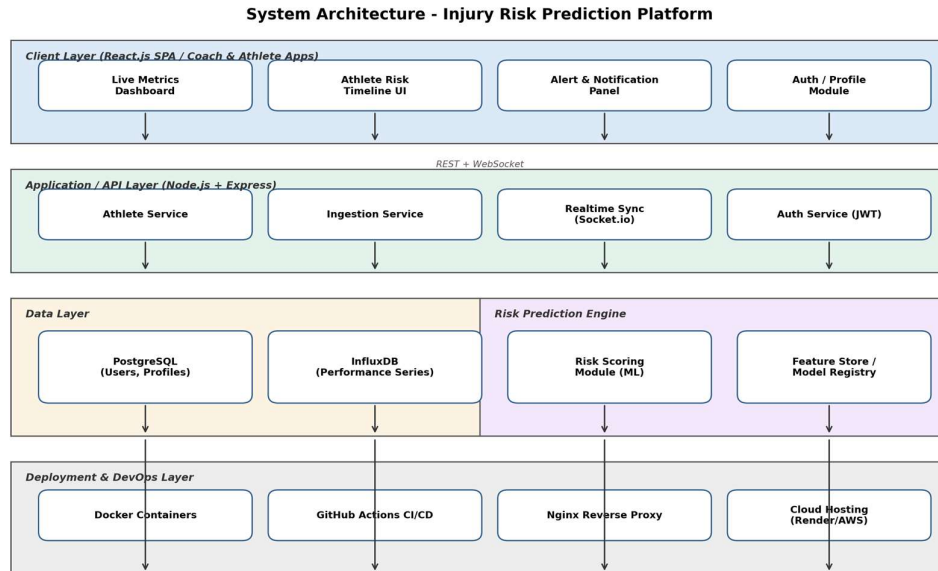


Figure 7.1 - High-level system architecture

Figure 7.1 - High-level system architecture

The Client Layer is a React single-page application responsible only for presentation and local dashboard state. The Application Layer exposes REST endpoints for CRUD operations and a WebSocket channel for real-time risk updates; it works alongside the dedicated Risk Prediction Engine, which is intentionally kept separate from the generic Athlete Service so that model logic can evolve independently. The Data Layer persists athlete/user metadata in PostgreSQL and the full history of performance readings and risk snapshots in InfluxDB. The Deployment Layer containerizes every service with Docker and fronts them with an Nginx reverse proxy.

7.4 Component Interaction and Quality Attribute Analysis

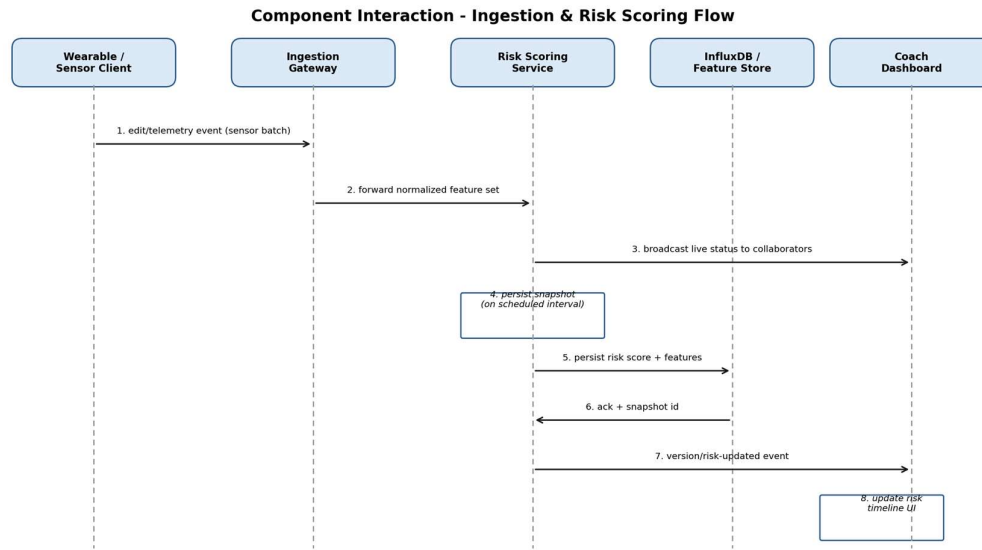


Figure 7.2 - Component interaction for the ingestion & risk-scoring flow

Figure 7.2 - Component interaction for the ingestion & risk-scoring flow

When a wearable device streams a batch of readings, the Ingestion Gateway normalizes and forwards the feature set to the Risk Scoring Service, which both broadcasts a live status update to connected coach dashboards (for the Real-Time / Availability quality attribute) and persists the computed score to the feature store. On a successful write, the store returns a snapshot id, which the service relays back to all connected clients so that the Risk Timeline UI updates for everyone - addressing the Consistency quality attribute by ensuring every collaborator sees the same risk history regardless of which device triggered the update.

Quality Attribute	How the Architecture Addresses It
Performance / Latency	WebSocket transport avoids polling overhead; feature events are small and batched.
Durability	Risk snapshots are immutable and persisted before an acknowledgement is sent to the client.
Scalability	Stateless API instances behind Nginx allow horizontal scaling; Socket.io supports a Redis adapter for multi-instance pub/sub.
Maintainability	Bounded service modules keep the Risk Prediction Engine independently testable and replaceable.
Security	JWT-based auth and role checks are enforced centrally in Express middleware before reaching any service.

Table 7.1 - Quality attribute analysis

7.5 Architectural Justification and Technical Presentation

The layered-plus-ML-service architecture was justified primarily by the durability and consistency requirements of a risk-history product: an architecture that could lose an in-flight reading or silently drop a computed score would defeat the core value proposition of the application. Persisting snapshots before acknowledging the client, combined with an append-only data model, was chosen over an in-place update model specifically because it makes data loss and inconsistent trend charts structurally harder to introduce,

at the acceptable cost of additional storage for historical snapshots - a trade-off explicitly discussed and accepted during design review.

7.6 Deployment Considerations

The containerized services are deployed behind an Nginx reverse proxy that terminates TLS and routes /api and /socket.io traffic to the server container while serving the built React bundle directly for all other paths. Environment-specific configuration (database URIs, JWT secret, allowed CORS origins, model version) is injected at container start via environment variables rather than baked into the image, so the same image can be promoted from staging to production without a rebuild. Database backups are scheduled independently of the application's own risk-history feature, since the two serve different purposes: application-level snapshots protect coaching staff from losing a computed score, while infrastructure backups protect against catastrophic data-layer failure.

8. Code Review Findings and Recommendations

8.1 Overall Findings Summary

Overall, the repository demonstrates a mature engineering process for a student project: a sensible layered architecture with a clearly bounded ML service, an enforced branching and code-review workflow, automated testing wired into CI, and documentation that is largely accurate and current. The most significant strength is the alignment between the domain problem (early, data-driven injury risk warning) and the engineering approach (immutable, append-only risk snapshots plus Git-based version control for the code itself), which shows the author applying the course's CO3 concepts consistently rather than treating them as separate boxes to tick.

8.2 Recommendations

8.2.1 Code Quality

- Add PropTypes or migrate to TypeScript for full compile-time type safety across client components.
- Split controllers exceeding ~150 lines into smaller, single-purpose service functions.
- Increase documentation coverage on client-side custom hooks to match the ml-service standard.

8.2.2 Repository Management

- Move trained model artefacts out of version control and into a dedicated model registry or Git LFS.
- Rename utility scripts under /server/scripts to follow the established camelCase-with-suffix convention.
- Add CODEOWNERS so that Pull Requests touching the Risk Prediction Engine automatically request review from the relevant maintainer.

8.2.3 Documentation

- Document Socket.io event contracts alongside the existing REST API reference.
- Add a troubleshooting section to the README covering common local Docker issues.

8.2.4 Testing

- Raise client-side statement coverage from 71% toward the 80%+ achieved on the server and ml-service.
- Add load tests simulating a higher number of simultaneously monitored athletes to validate the ingestion layer's scalability assumptions.

8.2.5 Future Enhancements

- Offline buffering on the device gateway with automatic replay on reconnect for training grounds with unreliable connectivity.

- Explainable-AI overlays on the risk score so coaches can see which specific factors (load spike, poor sleep, prior injury) are driving a given alert.
- Redis-backed Socket.io adapter to support horizontal scaling of the real-time layer in production.

8.3 Rubric Alignment Summary

The table below maps each evaluation criterion from the assessment rubric to the section of this report where it is addressed, to assist verification during grading.

Rubric Criterion	Addressed In
Problem Analysis & Repository Organization	Section 1 (Problem Overview), Section 2 (Repository Organization)
Code Quality Review	Section 3 (Code Quality Review)
Version Control Evaluation	Section 4 (Version Control Evaluation)
Testing & Validation	Section 5 (Code Testing and Validation)
Documentation & Repository Maintenance	Section 6 (Repository Documentation)
Review Findings, Recommendations & Presentation	Section 8 (Findings and Recommendations)
System Requirement Analysis	Section 7.1
Selection of Software Architecture	Section 7.2
Architecture Diagram and Component Design	Section 7.3
Component Interaction and Quality Attribute Analysis	Section 7.4
Architectural Justification and Technical Presentation	Section 7.5

Table 8.1 - Rubric-to-section alignment

9. Conclusion

The Injury Risk Prediction System for Athletes Using Performance Data successfully satisfies the CO3 requirement of building a full-stack application with a clear emphasis on containerization, version control, and collaborative development. The repository is well organized, the codebase follows consistent and largely well-documented coding standards across the Node.js, React, and Python components, the Git workflow supports safe collaborative development through branch protection and mandatory reviews, and an automated test suite integrated into CI provides confidence that changes do not silently break the risk-history guarantees that are central to the product. The recommendations above are incremental refinements rather than corrections to fundamental flaws, and addressing them would move the project from a solid student submission toward a production-ready sports-science tool.